

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ — ĐHQG HÀ NỘI (UET)

Báo cáo Bài tập lớn môn Lập trình Hướng đối tượng (OOP)



HỆ THỐNG ĐẤU GIÁ REAL-TIME

Auction System — Client-Server Desktop Application

THÀNH VIÊN THỰC HIỆN — NHÓM 7

Lê Công Xuân Phúc ★
MSSV: 25023358

Hồ Anh Tuấn
MSSV: 25023384

Nguyễn Anh Tùng
MSSV: 25023389

Trần Thanh Nhật
MSSV: 25023348

Tháng 06/2026

1. Mục tiêu & Phạm vi Dự án

MỤC TIÊU & CÔNG NGHỆ

Mục tiêu dự án

- ✦ Xây dựng hệ thống **đấu giá trực tuyến real-time** hoạt động mượt mà theo mô hình Client-Server.
- ✦ Áp dụng chặt chẽ **4 trụ cột OOP** (Kế thừa, Đa hình, Đóng gói, Trừu tượng) vào kiến trúc phần mềm thực tế.
- ✦ Sử dụng thành thạo các **Design Patterns** thực tế (Factory, Observer, Singleton, Strategy, Template Method).
- ✦ Giải quyết bài toán **đa luồng (Concurrency)** và đảm bảo an toàn luồng dữ liệu (Thread-safety).
- ✦ Tích hợp kiểm thử tự động, chuẩn hóa mã nguồn và quy trình **CI/CD hiện đại**.

Phạm vi & Công nghệ

- ✦ **Tính năng chính:** Multi-role (Bidder/Seller/Admin), đấu giá thời gian thực thông qua Socket, Auto-bidding tự động, Anti-sniping gia hạn giờ cuối, Dashboard quản trị.
- ✦ **Database:** Lưu trữ bền vững dữ liệu tài khoản, lịch sử giao dịch và sản phẩm bằng SQLite + JDBC.

CÔNG NGHỆ CHỦ CHỐT:

 Java 17 +

 JavaFX 17

 TCP Socket

 SQLite + JDBC

 Bcrypt

 Maven

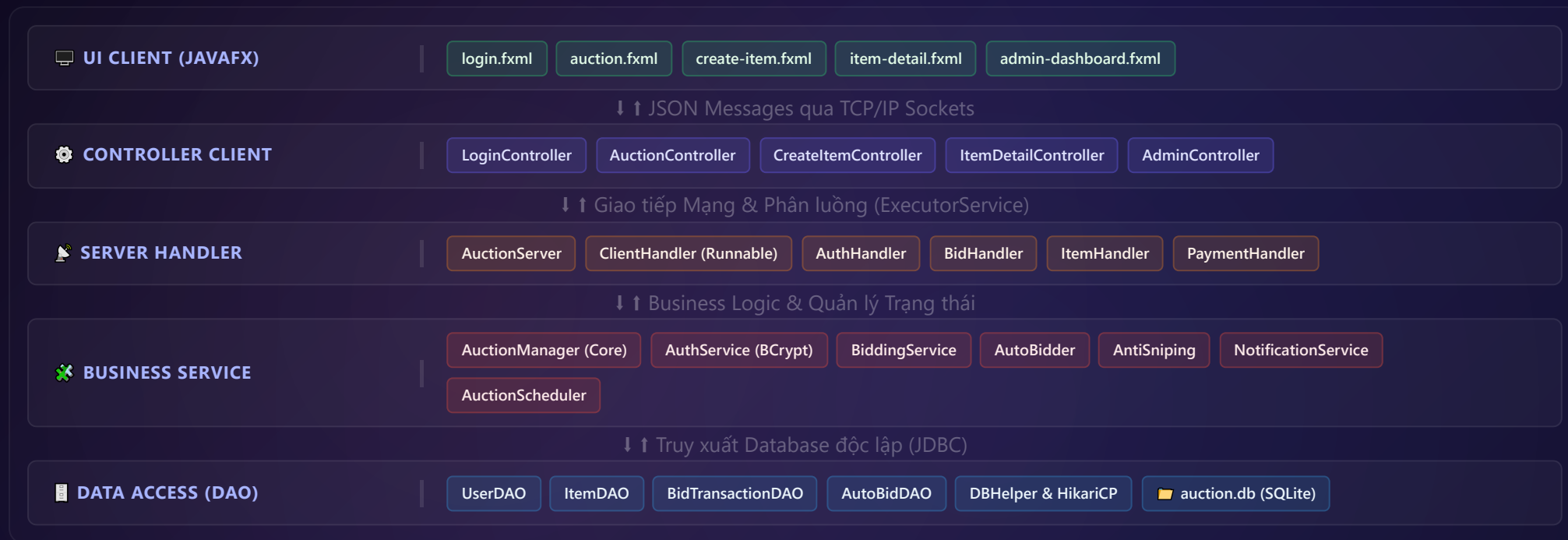
 Jackson JSON

 JUnit 5

 GitHub Actions

2. Kiến trúc Tổng thể Hệ thống

KIẾN TRÚC 5 TẦNG



3. Áp dụng 5 Design Patterns mẫu mực

Factory Pattern

Lớp: `ItemFactory`

Vấn đề: Tạo động các loại sản phẩm (Electronics, Art, Vehicle) có thuộc tính khác nhau.

Giải pháp: `createItem(type, ...)` nhận chuỗi loại và trả về lớp con tương ứng của `Item`.

Ý nghĩa: Đóng gói logic tạo đối tượng, tuân thủ chặt chẽ Open/Closed Principle.

Observer Pattern

Lớp: `BidObserver`

Vấn đề: Cập nhật trạng thái thời gian thực và gửi thông báo tới người theo dõi khi có bid mới.

Giải pháp: `Bidder` implements `BidObserver`. Lớp `Notification` quản lý và tự động broadcast.

Ý nghĩa: Giảm liên kết lỏng lẻo giữa luồng đấu giá và hệ thống gửi thông báo.

Singleton Pattern

Lớp: `AntiSniping`

Vấn đề: Đảm bảo chỉ có một luồng duy nhất đồng bộ và theo dõi thời gian kết thúc phiên đấu giá.

Giải pháp: Áp dụng Double-checked locking Singleton kết hợp thuộc tính `volatile` an toàn đa luồng.

Ý nghĩa: Tránh trùng lặp tài nguyên hệ thống và xung đột ghi đè thời gian.

Strategy Pattern

Lớp: `BiddingStrategy`

Vấn đề: Linh hoạt thay đổi quy tắc kiểm duyệt tính hợp lệ của giá đấu mới đặt.

Giải pháp: Khai báo interface `BiddingStrategy` với các lớp cụ thể (Default, Percentage, MinIncrement).

Ý nghĩa: Dễ dàng thay thế quy tắc kinh doanh mà không ảnh hưởng tới code lỗi.

Template Method

Lớp: `Item.java`

Vấn đề: Các lớp con sản phẩm có chung một quy trình đặt giá nhưng khác biệt logic chuyên biệt.

Giải pháp: Lớp cha abstract định hình khung `updateHighestBid()`, các lớp con override hooks mở rộng.

Ý nghĩa: Tái sử dụng tối đa code, tránh lặp mã nguồn giữa các phân loại.

4. Chức năng Nâng cao & Concurrent

CONCURRENT & SECURITY

⚡ Xử lý Đa luồng & Concurrency

- ♦ **Thread Pool:** Sử dụng `ExecutorService` (`CachedThreadPool`) phía Server để tiếp nhận hàng trăm Client kết nối đồng thời qua Socket mà không treo nghẽn hệ thống.
- ♦ **Thread-safe Collections:** Sử dụng `ConcurrentHashMap` cho dữ liệu phiên đấu giá và `CopyOnWriteArrayList` cho danh sách kết nối client.
- ♦ **Đồng bộ hóa:** Lock chuyên biệt bằng cơ chế `synchronized` kết hợp `ReentrantLock` tại các vùng trọng yếu để ngăn chặn lỗi tranh chấp dữ liệu (Race Conditions).

🛡️ Đấu giá Thông minh & Bảo mật

- ♦ **Auto-Bidding (Đấu giá tự động):** Cho phép người dùng thiết lập mức giá thầu trần mong muốn. Hệ thống tự động đặt giá tăng dần để duy trì vị thế dẫn đầu an toàn và bảo mật.
- ♦ **Anti-Sniping (Chống thầu chụp giờ cuối):** Phát hiện giá thầu đặt trong 10 giây cuối cùng, tự động **gia hạn thêm 20 giây** thời gian kết thúc của phiên, triệt tiêu sniping thầu chụp giật.
- ♦ **Bảo mật mật khẩu:** Mã hóa mật khẩu một chiều cực kỳ an toàn bằng thư viện chuyên dụng `**BCrypt**` trước khi ghi vào SQLite database.
- ♦ **Quản lý ngoại lệ chặt chẽ:** Định nghĩa bộ Custom Exceptions (`InvalidBid`, `ClosedAuction`) kết hợp với Unit Tests Junit 5 đạt tỷ lệ phủ mã nguồn cao.

5. Phân chia Công việc & Tổng kết

PHÂN CÔNG CÔNG VIỆC

Lê Công Xuân Phúc

NHÓM TRƯỞNG — MSSV: 25023358

- Thiết kế kiến trúc Socket Client-Server
- Xây dựng luồng Server, Handler Layer
- Broadcast real-time messages (JSON)
- Thiết lập CI/CD GitHub Actions

Server ClientHandler CI/CD

Hồ Anh Tuấn

CORE ENTITY & DB — MSSV: 25023384

- Thiết kế OOP Entity Hierarchy (Item, User)
- Tích hợp Factory & Observer Patterns
- Xây dựng cơ sở dữ liệu SQLite & JDBC
- Phát triển tầng DAO (UserDAO, ItemDAO)

Entity DAO SQLite

Nguyễn Anh Tùng

CORE SERVICE & TEST — MSSV: 25023389

- Phát triển Bidding, Auth & Strategy Services
- Hiện thực hóa Auto-Bidding & Anti-Sniping
- Xử lý Concurrency & Thread-safety lỗi
- Viết Unit Tests kiểm thử chất lượng (JUnit 5)

Service AutoBid JUnit 5

Trần Thanh Nhật

UI & CONTROLLER — MSSV: 25023348

- Thiết kế toàn bộ UI FXML & CSS (JavaFX)
- Xây dựng Controller Layer liên kết UI
- Phát triển Notification & Analytics Services
- Hiện thực hóa thanh toán (PaymentHandler)

JavaFX UI/CSS Payment

📋 BAREM ĐIỂM HOÀN THÀNH XUẤT SẮC

- ♦ Kế thừa & Kín đóng gói (OOP 4 trụ cột)
 - ♦ Đa hình dynamic & Trừu tượng hóa cao
- ♦ 5 Design Patterns mẫu mực
 - ♦ Socket đa người chơi thời gian thực
- ♦ Thread-safe Concurrency & Auto-Bid
 - ♦ Bảo mật BCrypt & CI/CD GitHub Actions